Week 4: The Language of Change

Linear Algebra I, Linear Transformations, Rotation, and MATLAB Basics

Instructor / Mentor

Kiki Research Training Program

Week 4

Contents

From Equations to Vector Spaces

Why Revisit Linear Algebra?

Last time, we used the matrix inverse to solve a linear system such as $Ax = b$.

Today we move one level deeper

- A matrix is not only a table of numbers.
- It represents an **operation** acting on vectors.
- Linear algebra gives us a precise language for **change**, **motion**, and **coupling**.

Big picture

In science and engineering, many models become easier once we ask: *what space are we working in, and what transformation acts on that space?*

What Is a Vector?

Formal view

A **vector** is an element of a vector space. In coordinates, we often write

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n.$$

- It can represent position, velocity, current injection, features, or a signal.
- The arrow picture is helpful, but the definition is more general than geometry.
- What matters is that vectors can be **added** and **scaled**.

Examples

- 2D point: $\begin{bmatrix} x \\ y \end{bmatrix}$
- 3-phase current readings
- A sampled time series
- Pixel intensities in an image patch

What Is a Linear Space?

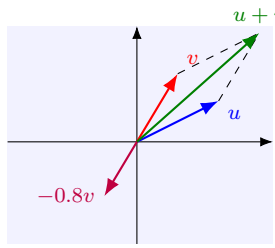
Mathematical definition

A **linear space** (or **vector space**) over a field such as $\mathbb{R}$ is a set V with two operations: vector addition $u + v$ and scalar multiplication αv, such that the usual rules hold.

Key rules we rely on:

- closure under addition and scaling
- usual algebra rules for addition
- existence of a zero vector and negatives
- distributive rules: $\alpha(u + v) = \alpha u + \alpha v$

Examples: $\mathbb{R}^n$, polynomials, matrices. *Not a vector space:* only positive vectors.



Closure picture

Add arrows or scale them, and you still stay in the same space.

Span, Basis, and Coordinates

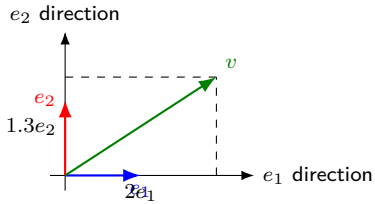
Three words we need today

- **Span:** all linear combinations of chosen vectors.
- **Basis:** linearly independent vectors that span the space.
- **Coordinates:** the coefficients relative to that basis.

In $\mathbb{R}^2$, the standard basis is

$$e_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad e_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

A matrix is the coordinate description of a transformation after we choose bases.



Coordinate picture

Coordinates tell us how much of each basis vector is needed to rebuild v:

$$v = 2e_1 + 1.3e_2.$$

Linear Transformations and Matrices

From Mapping to Linear Transformation

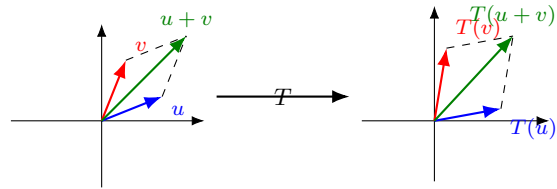
General mapping

A mapping $T : V \to W$ sends an input vector from one linear space to an output vector in another.

Definition of linear transformation

The mapping T is **linear** if for all vectors $u, v \in V$ and scalars α, β,

$$T(\alpha u + \beta v) = \alpha T(u) + \beta T(v).$$



- Preserve addition: map together or separately, then add.
- Preserve scaling: double input, double output.
- Therefore every linear transformation satisfies $T(0) = 0$.

How Do We Get a Matrix from a Linear Transformation?

1. Choose a basis $E = (e_1, \ldots, e_n)$ for the input space V.
2. Choose a basis $F = (f_1, \ldots, f_m)$ for the output space W.
3. Compute the image of each input basis vector:

$$T(e_j) = a_{1j} f_1 + a_{2j} f_2 + \cdots + a_{mj} f_m.$$

4. Use those coordinate vectors as the columns of a matrix:

$$A = \begin{bmatrix} [T(e_1)]_F & [T(e_2)]_F & \cdots & [T(e_n)]_F \end{bmatrix}.$$

5. Then every vector v satisfies

$$[T(v)]_F = A[v]_E.$$

Interpretation

The matrix is not arbitrary. Its columns are exactly **where the basis vectors go**.

Worked Example: Neighborhood Demand → Feeders

Input and output spaces

$x = \begin{bmatrix} x_N \\ x_S \end{bmatrix}$ = extra demand in the north and south neighborhoods (kW).

$y = T(x) = \begin{bmatrix} y_A \\ y_B \end{bmatrix}$ = feeder-meter readings (kW).

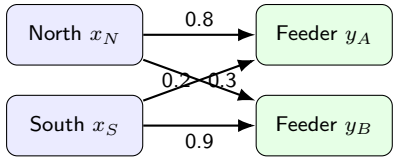
Suppose one extra unit of demand produces

$$T(e_1) = \begin{bmatrix} 0.8 \\ 0.2 \end{bmatrix}, \qquad T(e_2) = \begin{bmatrix} 0.3 \\ 0.9 \end{bmatrix}.$$

So the matrix is

$$A = \begin{bmatrix} 0.8 & 0.3 \\ 0.2 & 0.9 \end{bmatrix}$$

So each feeder reading is a weighted sum of neighborhood demand.



Physical meaning of linearity

Addition: combine two demand scenarios, and the feeder totals add.
Scaling: double every demand input, and every feeder reading doubles.

Another Everyday Example: Scores → Parents' Happiness

Input and output spaces

$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ = the math scores of child 1 and child 2.

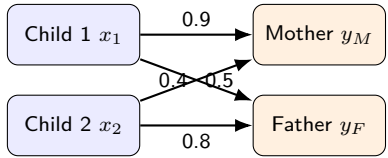
$y = T(x) = \begin{bmatrix} y_M \\ y_F \end{bmatrix}$ = the happiness levels of mother and father.

Suppose each parent reacts differently to each child's score:

$$T(e_1) = \begin{bmatrix} 0.9 \\ 0.4 \end{bmatrix}, \qquad T(e_2) = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}.$$

Then the matrix is

$$A = \begin{bmatrix} 0.9 & 0.5 \\ 0.4 & 0.8 \end{bmatrix}$$



Why linearity still makes sense

Addition: if both children improve their scores, the parents' happiness contributions add together.
Scaling: if both scores improve by twice as much, the happiness change is twice as large.

Why Does Matrix-Vector Multiplication Mean "Apply the Transformation"?

Let the columns of A be $a_1, a_2, \ldots, a_n$. Then

$$Ax = x_1a_1 + x_2a_2 + \cdots + x_na_n.$$

Meaning

If the input vector is written in coordinates as $x = [x_1, x_2, \ldots, x_n]^T$, then the output is the same linear combination of the **transformed basis vectors**.

Connection to last week

In $Ax = b$, the matrix A is a machine. Solving the system means finding the input x that produces the observed output b.

Rotation and Trigonometry

Sine and Cosine: The Geometry We Need

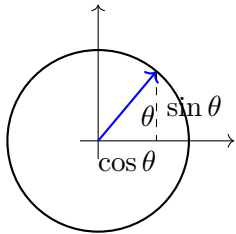
Unit-circle definition

On the unit circle, the point at angle θ has coordinates

$$(\cos\theta, \sin\theta).$$

So:

- $\cos\theta$ = horizontal coordinate
- $\sin\theta$ = vertical coordinate



Right-triangle reminder

$$\cos\theta = \frac{\text{adjacent}}{\text{hypotenuse}}, \qquad \sin\theta = \frac{\text{opposite}}{\text{hypotenuse}}.$$

Deriving the Counterclockwise Rotation Matrix

Start with a vector of length r making angle ϕ with the positive x-axis:

$$v = \begin{bmatrix} r\cos\phi \\ r\sin\phi \end{bmatrix}.$$

After rotating by angle θ counterclockwise, the new angle is $\phi + \theta$:

$$v' = \begin{bmatrix} r\cos(\phi+\theta) \\ r\sin(\phi+\theta) \end{bmatrix}.$$

Using the angle-addition formulas,

$$\cos(\phi+\theta) = \cos\phi\cos\theta - \sin\phi\sin\theta,$$

$$\sin(\phi+\theta) = \sin\phi\cos\theta + \cos\phi\sin\theta,$$

we get

$$v' = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} r\cos\phi \\ r\sin\phi \end{bmatrix}.$$

Rotation Matrices: Counterclockwise and Clockwise

Counterclockwise by θ

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Clockwise by θ

$$R(-\theta) = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Quick example

Rotating $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ by 90° counterclockwise gives

$$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

MATLAB Basics and vibe coding

MATLAB Basics: Just Enough to Start

Core syntax ideas

- matrices use square brackets
- spaces or commas separate columns
- semicolons separate rows
- indexing starts from **1**, not 0
- `*` is matrix multiplication
- `.*` is elementwise multiplication

Good habits

- use descriptive variable names
- display intermediate results
- plot what you compute
- check matrix dimensions before multiplying
- prefer solving systems with `A\b`

How to Find MATLAB Functions Fast

Three reliable ways

1. Use the **MATLAB Help Center / MathWorks documentation** and search by task.
2. In MATLAB, try `doc functionName` for full documentation.
3. Try `help functionName` for a quick command-line summary.

What to search for

Search with intent, not only function names:

- "solve linear system MATLAB"
- "MATLAB plot vector with quiver"
- "MATLAB cos sin degrees radians"

Student mindset

Read the example, change the numbers, then test it yourself.

vibe coding Task 1: Solve a Linear System in MATLAB

Goal: experience matrix-native computation in MATLAB using a small microgrid-style system.

Prompting hints for your AI assistant

1. Ask for a MATLAB script that defines a small matrix A and vector b for a 3-node or 5-node linear system.
2. Ask it to solve the system using MATLAB's system-solving operator, not manual Gaussian elimination.
3. Ask it to print the solution and explain the physical meaning of each variable.
4. Ask one follow-up question: why is solving with `A\b` usually better than explicitly forming the inverse?

Do not copy blindly. Check dimensions, units, and whether the result is reasonable.

vibe coding Task 2: Rotate and Visualize a Vector

Goal: turn the rotation matrix into a visible geometric action.

Prompting hints for your AI assistant

1. Ask for a MATLAB script that defines one 2D vector and one rotation angle θ.
2. Ask it to build the rotation matrix from `cos(theta)` and `sin(theta)`.
3. Ask it to compute the rotated vector using matrix multiplication.
4. Ask it to draw the original and rotated vectors on the same axes.
5. Ask it to let you change θ easily so you can test clockwise and counterclockwise cases.

Hint: if the picture looks wrong, first check whether your angle is in radians or degrees.

vibe coding Task 3: Explore Translation with an Extra Coordinate

Goal: see why translation needs one more dimension if we want a matrix form.

Prompting hints for your AI assistant

1. Ask for a MATLAB example that translates a 2D point by (t_x, t_y).
2. Ask it to show why ordinary 2×2 matrix multiplication cannot produce this translation by itself.
3. Ask it to augment the point into homogeneous coordinates $[x, y, 1]^T$.
4. Ask it to construct the corresponding 3×3 transformation matrix and visualize before/after points.

This is the bridge to computer vision, robotics, motion control, and path planning.

Beyond Linear: Affine Transformations

Why Translation Is Not Linear in Ordinary Euclidean Space

Consider the mapping

$$T(x) = x + t,$$

where t is a fixed translation vector.

Test 1: zero vector

A linear map must satisfy $T(0) = 0$. But here,

$$T(0) = t \neq 0$$

unless $t = 0$.

Conclusion

Translation is generally **not** a linear transformation on $\mathbb{R}^n$. It is an **affine transformation**.

Homogeneous Coordinates: Turning Affine into Matrix Form

In 2D, replace the point $\begin{bmatrix} x \\ y \end{bmatrix}$ by the augmented vector

$$\tilde{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}.$$

Then translation by (t_x, t_y) can be written as

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}.$$

Why this matters

Rotation, scaling, and translation can now share one matrix framework. This is standard in computer vision, robotics, graphics, and motion planning.

Homework

Homework for This Week

1. Write a MATLAB script to complete matrix-vector multiplication manually using loops:
$$y(i) = y(i) + A(i, j)\, x(j).$$
2. In one paragraph, explain why the columns of a matrix tell us where the basis vectors go.
3. Rotate one 2D vector by two different angles and explain the geometric difference.
4. Bonus: explain why translation is affine, not linear, and how homogeneous coordinates fix the representation problem.

Thank you!

Next week: Calculus intuition, rates of change, and accumulation.